AI-powered Virtual Humans

Part 1: Virtual Humans Foundations



Day 2

Stefano Calzolari · Wed 21 – Fri 23 October 2026

The system we build: the body works, now the brain

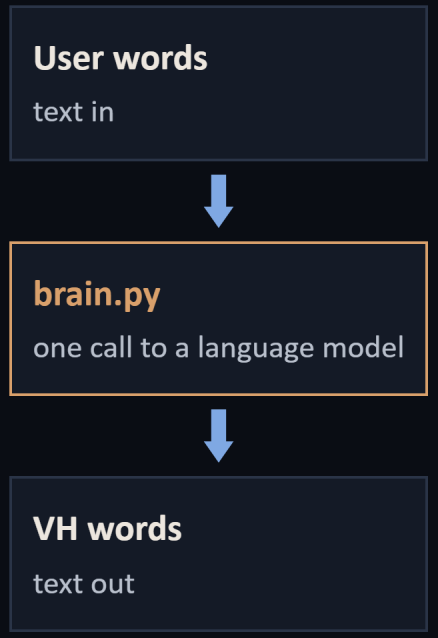


01

An LLM brain in Python

Why not run the model inside Unity?

What is the simplest brain that talks?



Why the brain lives in Python?

Swap the model

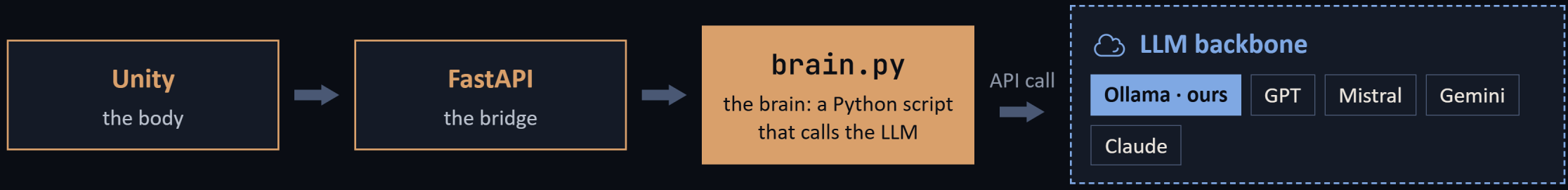
Any provider, any size, without touching the body

The Python ecosystem

LangChain and LangGraph live here: Part 2 builds on them

Rendering apart from inference

Unity draws frames; the model thinks elsewhere



True **for an ECA**: turn-based, one answer per question, so the brain can think elsewhere

Continuous interaction, like **Max** assembling with the user, needs perception, reaction and deliberation in the loop all the time: **we would need to rethink the architecture**

Python environment setup

Terminal, in the repository

```
cd PythonProject
python -m venv .venv
#
# Windows:
.venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate
#
pip install -r requirements.txt
```

requirements.txt

```
fastapi
uvicorn[standard]
python-multipart
pydantic⩾2
python-dotenv
ollama
faster-whisper
av==13.1.0
piper-tts==1.8.0
# Windows, NVIDIA GPU:
nvidia-cublas-cu12
nvidia-cudnn-cu12
```

FastAPI is for the bridge; everything else is used today. The speech models are not downloaded: copy them into PythonProject/models/. The two nvidia lines (about 1 GB) let Whisper use a GPU; without one it runs on the CPU.

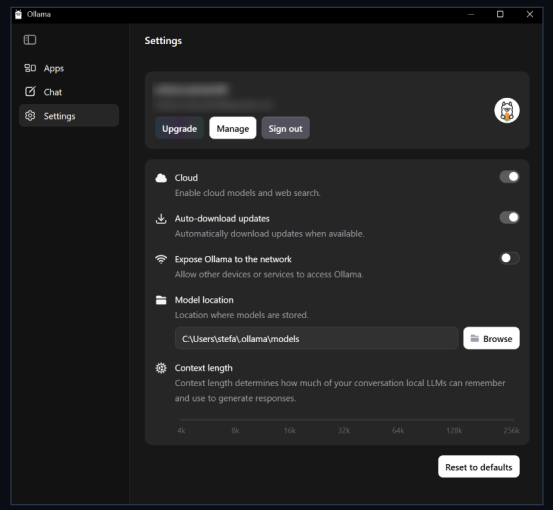
Using the Ollama app

Terminal, any folder

```
ollama signin
# sign in to your ollama.com account
ollama run gpt-oss:120b-cloud
# try a model: chat, /bye to exit
ollama pull llama3.2:3b
# optional: download a model to run locally
ollama list
# models available on this machine
ollama ps
# models loaded right now
```

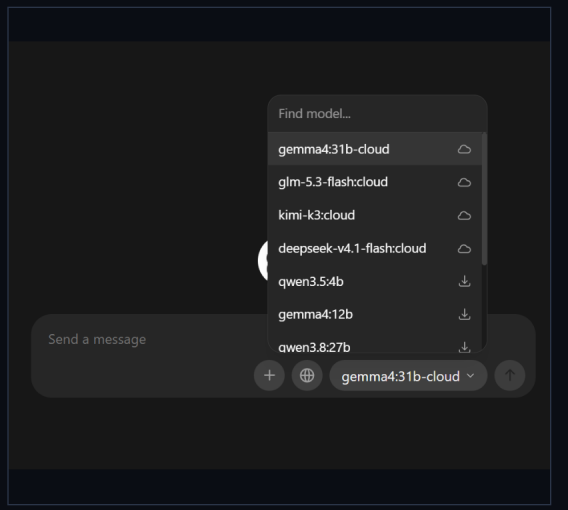
The app keeps a server on **localhost:11434**: brain.py talks to it.
Quit from the tray stops it, and the brain falls back to the echo

Or, in the app window



Settings

Sign in, keep **Cloud** on



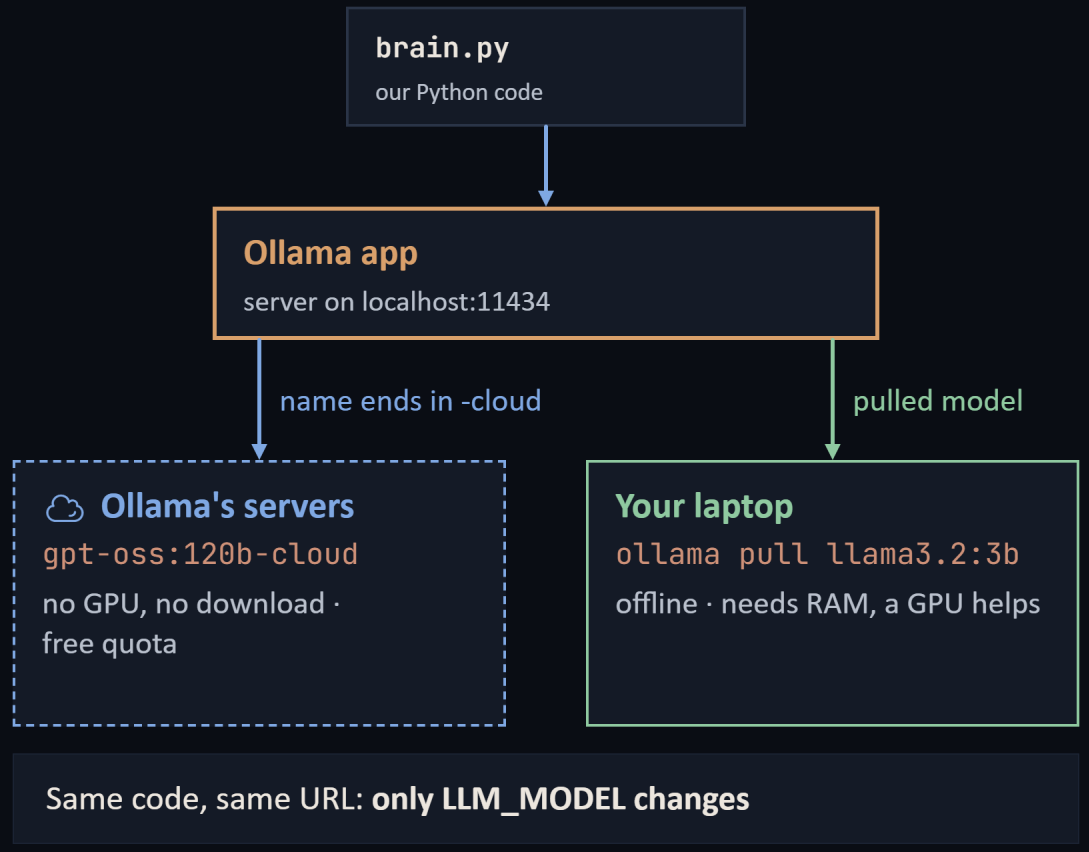
Chat → model menu

Cloud icon: runs on Ollama's servers.
Download icon: pulls it to your laptop.
Pick one and ask a question

Connect Python to Ollama

1. Install the **Ollama app**, sign in (free account)
2. Test: **ollama run gpt-oss:120b-cloud**
3. Copy **.env.example** to **.env**: the defaults already work, no key to paste

```
# PythonProject/.env  (defaults, commented)
# LLM_MODEL=gpt-oss:120b-cloud
# LLM_HOST=http://localhost:11434
# remove the # only to change one
```



brain.py: a simple LLM call

```
TEXT_PROMPT = """You are Camila, a friendly virtual assistant who lives
inside a 3D scene and talks with a person face to face.
Answer in one or two short sentences, in the language of the user.
Warm, natural and spoken; no lists, no markdown, no emojis."""

_HOST  = os.getenv("LLM_HOST", "http://localhost:11434")  # the Ollama app
_MODEL = os.getenv("LLM_MODEL", "gpt-oss:120b-cloud")     # Ollama's servers
_client = Client(host=_HOST)                               # app off → None → echo

def think(user_text: str) → BrainOutput:        # THE SEAM
    return BrainOutput(speech_text=think_text(user_text))
```

The prompt is **configuration**: persona + style

App not running → no client → the **echo**: never a crash

think(): the server only ever calls this

A chat is a list of messages

```
[
    {"role": "system",    "content": "You are Camila, …"},
    {"role": "user",      "content": "Hi, who are you?"},
    {"role": "assistant", "content": "I'm Camila! …"},
    {"role": "user",      "content": "What can you do?"}
]
```



Ollama turns the list into the model's own prompt format, with special tokens per role; the model only continues it.

The ChatGPT app

Keeps the history for you, adds its own hidden system prompt, memory and tools. You type only the next line.

Our LLM system

The model is **stateless**: every call sends the whole list. **We** write the system prompt and decide what history to keep.

LIVE-3: one call, text in, text out

```python
def think_text(user_text: str) -> str:
    if _client is None:
        return user_text                          # echo: no Ollama app
    # ===== LIVE-3 · Brain ▸ LLM =====
    try:
        resp = _client.chat(model=_MODEL, messages=[
            {"role": "system", "content": TEXT_PROMPT},
            {"role": "user",   "content": user_text}])
        return resp.message.content.strip()
    except Exception as e:                        # network, app, quota...
        print("Brain: LLM call failed:", e)
        return user_text
```

A chat = a list of **messages with roles**

system: who Camila is · **user**: what you said

Any failure → the echo

Try it: talk to the brain

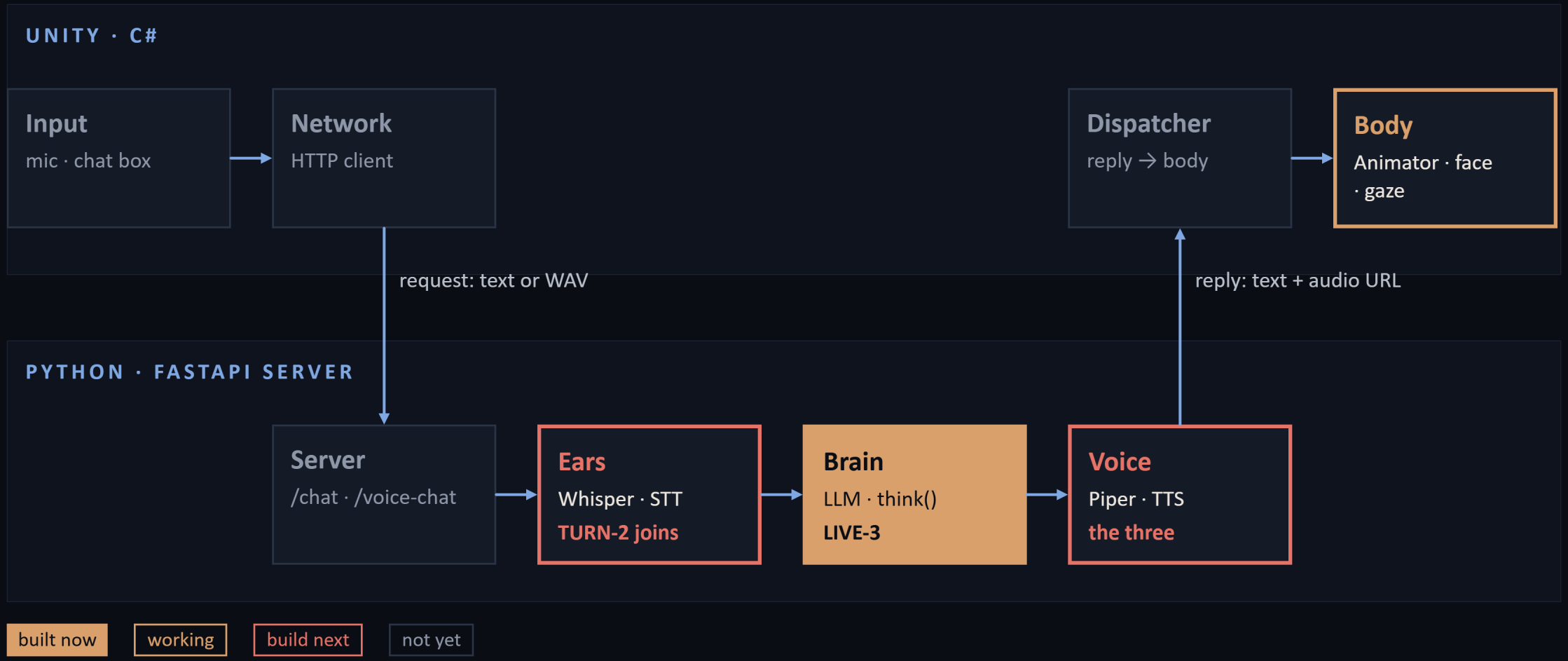
1. **python brain.py**
2. Ask three things: a greeting, a fact, an opinion
3. Change the persona in the prompt, run again

> Answers longer than two sentences? Tighten the **style** lines

Troubleshooting

Symptom	Fix
ModuleNotFoundError	The venv is not active, or pip ran elsewhere
Always an echo of your own words	LIVE-3 not written, the Ollama app not running, or LLM_HOST not localhost:11434
401 or 403 in the log	App not signed in: sign in to ollama.com
429 or quota errors	Free quota used up: wait, or pull a local model
Very slow answers	A smaller cloud model in LLM_MODEL

Where we are: the brain answers

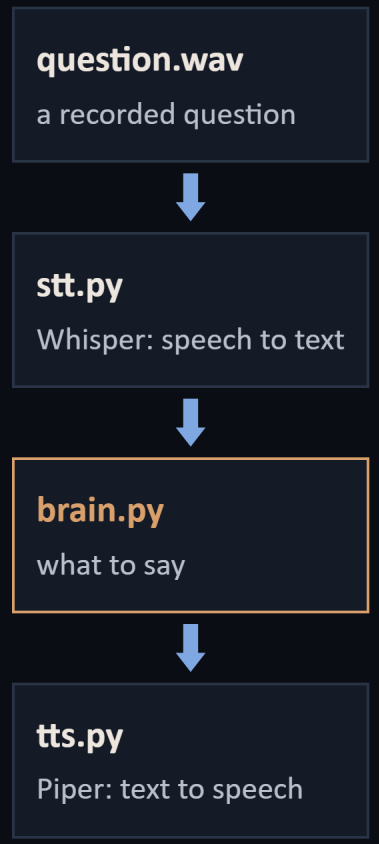


02

Ears and voice

How does audio become text, and back?

How long does each stage take?



Get the speech models: download_models.py

1. Open a terminal in the **repository root**: the folder that contains **download_models.py** and **VH_Starter_Project**
2. Run it

```
python download_models.py
```

3. Wait for **All models are in place.** Files already there are skipped

TROUBLESHOOTING

FAILED or interrupted → run it again: it resumes.
Still failing? → links in **PythonProject/models/README.md**

Lands in PythonProject/models/

Folder	Block	Size
faster-whisper-small	Ears	≈ 480 MB
piper-voice-en_US-hfc_female-medium	Voice	≈ 60 MB
piper-voice-it_IT-serena-high	Voice · optional	≈ 110 MB

Each Piper voice is **two files**: .onnx + .onnx.json. The script fetches both

Download once · then everything runs offline

Speech-To-Text: VH ears with Whisper

IN
Audio: what the user says
a WAV recording, any language

→

STT
speech to text

→

OUT
`"Hi, who are you?"`
plain text, ready for the brain

Model	Speed	Use it for
tiny	Fastest	Least accurate: quick tests
base	Fast on CPU	Clear English; more mistakes on short phrases
small	Fine on a laptop	Default for class: also good in Italian
medium / large	Needs a GPU	Best quality

Compute
int8 on the CPU, or on the GPU if present

Language
Auto-detect, or fixed (en, it) for short phrases

Local · free · no key · no account

Radford et al., Robust speech recognition via large-scale weak supervision (Whisper), OpenAI 2022 · faster-whisper
Links: arxiv.org/abs/2212.04356 · github.com/openai/whisper · github.com/SYSTRAN/faster-whisper

The ears: stt.py

```python
# local model folder: models/faster-whisper-small (no download)
_MODEL_PATH = os.getenv("WHISPER_MODEL") or "models/faster-whisper-small"
_LANGUAGE   = os.getenv("WHISPER_LANGUAGE") or None    # None = auto-detect
_DEVICE     = "cuda" if a GPU is there else "cpu"
_model = _load()                          # ONCE, at import: seconds

def transcribe(path: str) -> str:         # a sentence: a fraction of a second
    segments, _ = _model.transcribe(path, vad_filter=True, language=_LANGUAGE)
    return " ".join(s.text.strip() for s in segments).strip()

# test alone:  python stt.py Recording.m4a   (WAV works too)
```

In: a **WAV path** · Out: **text**

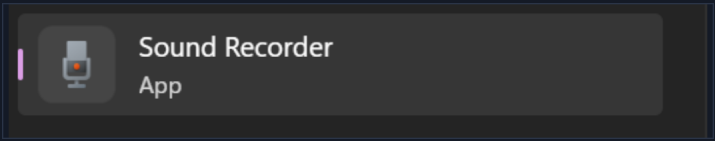
The model loads **once**, at import

vad_filter skips the silence

GPU if present, else CPU int8

TRY IT

1 Start menu → **Sound Recorder**



2 Record a short question: it lands in **Documents\Sound Recordings**

3 **python stt.py "...\Recording.m4a"**

Text-To-Speech: give the VH a voice

IN

`"I'm Camila, nice to meet you!"`

the brain's reply, plain text

→

TTS

text to speech

→

OUT

Audio: the VH's voice

a WAV file, mono 22 kHz, for Unity

Engine	Strengths	Cost and setup
Piper	Runs offline, many languages	Free
ElevenLabs	High quality, expressive	Paid API
Azure, Google, OpenAI TTS	High quality	Paid API
edge-tts	High quality (but unstable)	Free

Same block, any engine: swap Piper for another, nothing else changes

Unity wants **WAV**: Piper writes it directly, mono 22 kHz

Links: github.com/OHF-Voice/piper1-gpl · Piper voices · elevenlabs.io · github.com/rany2/edge-tts

The voice: tts.py

```python
# local voice: models/piper-voice-en_US-hfc_female-medium (no network)
LANGUAGE = os.getenv("TTS_LANGUAGE") or "en"      # en | it
_voice = PiperVoice.load(...)                       # ONCE, at import

async def synthesize(text: str) → str:
    """Returns a relative URL like /static/audio/<id>.wav"""
    wav = AUDIO_DIR / f"{uuid.uuid4().hex}.wav"
    await run_in_threadpool(_synth, text, wav)      # off the event loop
    return f"/static/audio/{wav.name}"

def cleanup(max_age_s=600): ...                     # delete old WAVs

# test alone:  python tts.py "Hello there"
```

In: **text** · Out: a **URL** to a WAV

Mono 22 kHz WAV: Unity plays it as is

static/audio: next hour the server serves it

TRY IT

1. **python tts.py "Hello, I am Camila"**
2. It prints **/static/audio/<id>.wav**: the file is in **PythonProject\static\audio**
3. Double-click the WAV: Camila's voice

YOUR TURN · TURN-2

TURN-2: join the three boxes

```python
# try_pipeline.py — the whole brain side, no Unity, no server
from stt import transcribe
from brain import think_text
from tts import synthesize

# ===== TURN-2 · Ears ▸ Brain ▸ Voice =====
# WAV in sys.argv[1] → transcribe → think_text → synthesize
# synthesize is async: asyncio.run(...)
# a timestamp t0..t3 around each call
heard, reply, audio = "", "", ""          # placeholder

print(f"stt {t1-t0:.2f} s · brain {t2-t1:.2f} s · tts {t3-t2:.2f} s")
```

Three calls, in order: the same chain the server will run

time.perf_counter() before and after each

Run: **python try_pipeline.py Recording.m4a**

TURN-2: solution try_pipeline.py

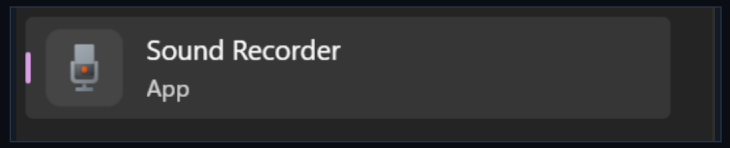
```python
# try_pipeline.py — the whole brain side, no Unity, no server
import asyncio, sys, time
from stt import transcribe
from brain import think_text
from tts import synthesize

# ===== TURN-2 · Ears ▸ Brain ▸ Voice =====
t0 = time.perf_counter(); heard = transcribe(sys.argv[1])
t1 = time.perf_counter(); reply = think_text(heard)
t2 = time.perf_counter(); audio = asyncio.run(synthesize(reply))
t3 = time.perf_counter()

print("Heard:", heard)
print("Reply:", reply)
print("Audio:", audio)
print(f"stt {t1-t0:.2f} s · brain {t2-t1:.2f} s · tts {t3-t2:.2f} s")
```

Try it: question in, voice out

1. Record a short question with **Sound Recorder** (on Windows)



2. **python try_pipeline.py "...\Recording.m4a"**
3. Play the WAV, write down the three times

Stage	s
STT - Ears	
LLM - Brain	
TTS - Voice	

Troubleshooting

Symptom	Fix
Empty transcript	Record again, closer to the mic
Wrong language	WHISPER_LANGUAGE=it or en in .env
FileNotFoundError: model	Copy the model folders into models/
FileNotFoundError: ….json	TTS_MODEL in .env points at a folder: comment it out
Only an echo comes back	LLM_HOST must be localhost:11434, Ollama app running
First run slow	The models load from disk: wait
The voice reads markdown	Tighten the style lines in the prompt

Both sides are ready

Two tracks done

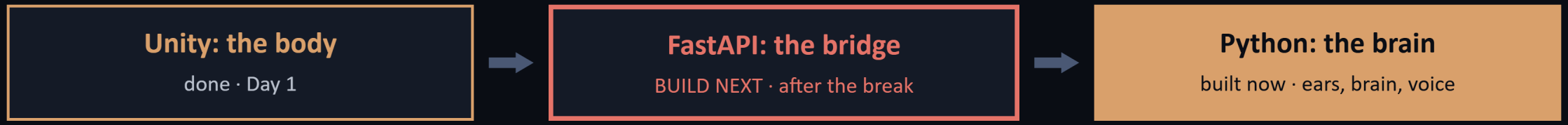
- ✓ **Unity**, the body: idle, blink, gaze, lip-sync (Day 1)
- ✓ **Python**, the brain: one LLM call in brain.py
- ✓ Ears: stt.py with Whisper
- ✓ Voice: tts.py with Piper
- ✓ The chain, timed: try_pipeline.py

Next: the bridge

FastAPI: the brain as a service

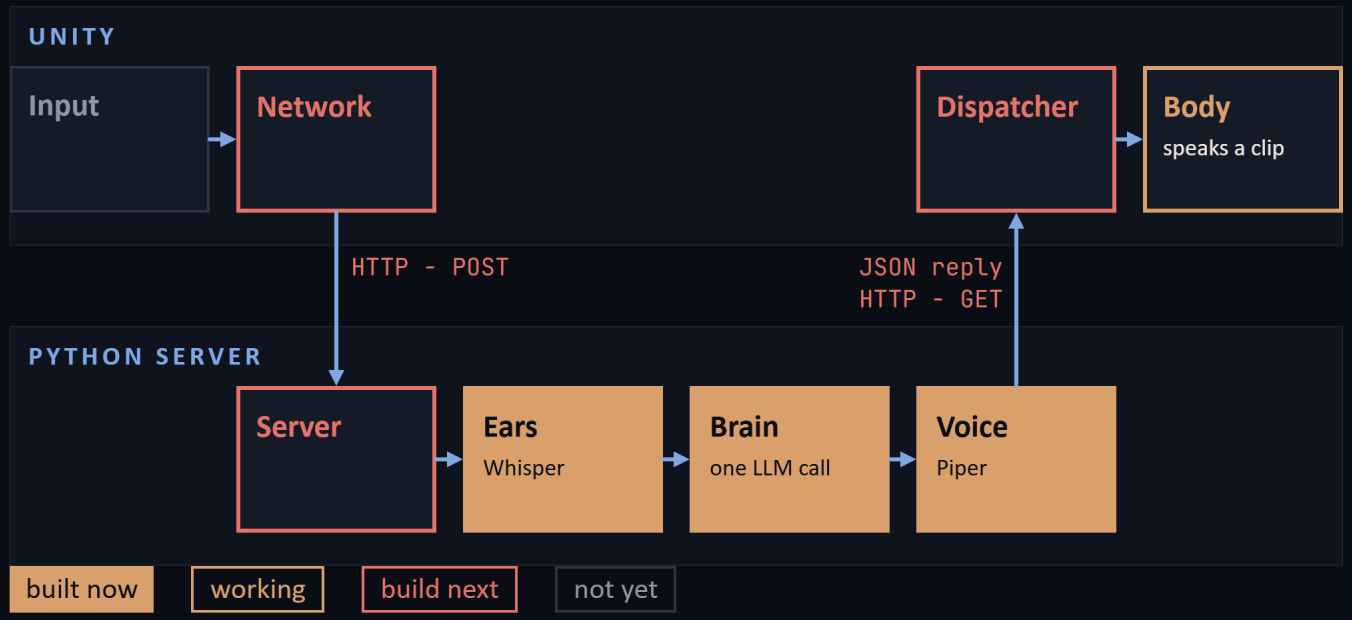
Unity calls it over HTTP

The character speaks the answer



built now | working | build next

What works now: brain and voice



✓ **Brain**
python brain.py answers in character

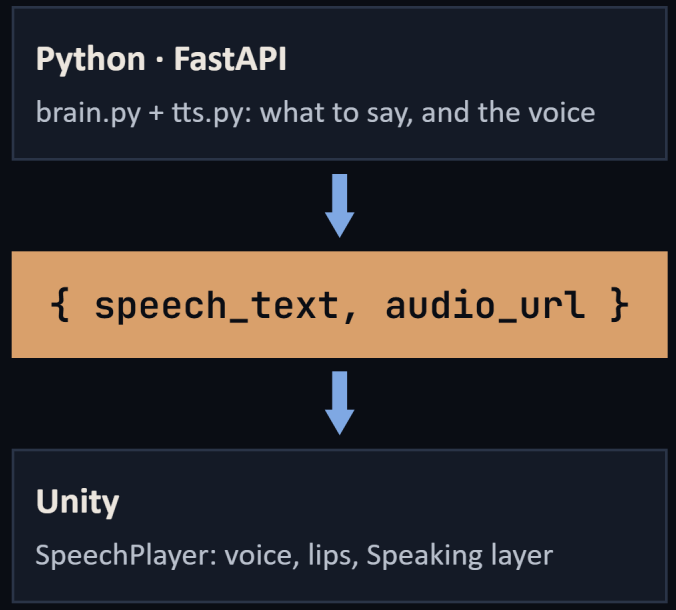
✓ **Ears and voice**
stt.py and tts.py work alone

✓ **Pipeline**
try_pipeline.py: three timings written down

03

The contract between Unity and Python

How do the two programs talk?



Why a bridge? SAIBA roles in code

Python (FastAPI)
Intent + behavior planner: decides **what**

→

`JSON payload`
the only coupling

→

Unity
Behavior realizer: decides **how**

The server can run on another machine

Test the brain in a browser, without Unity

Part 2 replaces the brain; Unity stays as it is

Swap the brain without touching the body.

Hartholt et al., Re-architecting the Virtual Human Toolkit, ACM IVA 2022 · Kopp et al., SAIBA

HTTP or WebSocket?

HTTP POST PART 1

Request → response: one turn of conversation

Testable alone in Swagger or Postman

Easy to debug

WebSocket UPGRADE PATH

One open connection, either side can send

Streaming audio and partial results

Lower latency, interruptions, backchannels

Start simple. The **JSON contract stays the same** when you change the transport.

The contract, version one

```
{
  "speech_text": "Hi! I'm Camila. What can I do for you?",
  "audio_url": "/static/audio/3f2a….wav"
}
```

Field	Meaning
speech_text	What the character says
audio_url	Where Unity downloads the voice

Grows in the webinar: **action** and **emotion**.

The contract, in Python and in C#

schemas.py

```python
class ChatRequest(BaseModel):
    text: str

class BrainOutput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    speech_text: str
    # TURN-5 (webinar): + action, emotion

class ChatResponse(BrainOutput):
    audio_url: Optional[str] = None
    transcript: Optional[str] = None
    timings_ms: Optional[Dict[str, int]] = None
    brain: Optional[str] = None  # "echo" or "llm"
```

AgentNetworkClient.cs

```csharp
[Serializable] public class ChatRequest
{ public string text; }

[Serializable] public class ChatResponse
{
    public string speech_text;
    public string audio_url;
    public string transcript;
    public Timings timings_ms;
    public string brain;
    public string action, emotion;   // webinar
}
```

main.py: the server around one function

```python
from stt import transcribe
from brain import think, brain_name
from tts import synthesize, cleanup

app = FastAPI(title="Virtual Human Server", lifespan=lifespan)
app.mount("/static", StaticFiles(directory="static"))   # WAVs by URL

@app.get("/")               # status + which brain
@app.post("/chat")          # text  → respond(text)
@app.post("/voice-chat")    # WAV   → transcribe → respond   (TURN-4)
@app.post("/stt-test")      # stage test for the ears

async def respond(text, transcript=None, t_stt_ms=0) → ChatResponse:
    # LIVE-4: think, then speak
```

Reuses the **ears, brain, voice** of the Python track

/static: Unity downloads the voice by URL

Both endpoints meet in **respond()**

LIVE-4: think, then speak

```python
async def respond(text, transcript=None, t_stt_ms=0):
    cleanup()
    t0 = time.perf_counter()
    # ===== LIVE-4 · Server ▸ Brain ▸ Voice =====
      out = await run_in_threadpool(think, text)  # blocking: off the loop
      t1 = time.perf_counter()
      try:
          audio_url = await synthesize(out.speech_text)
      except Exception as e:              # no voice: text still goes
          print("TTS failed:", e)
          audio_url = None
    t2 = time.perf_counter()
    return ChatResponse(**out.model_dump(),
                        audio_url=audio_url, ...)
```

Your **TURN-2** pipeline, as a service

The LLM call in a **thread pool**: the server stays responsive

No voice? The reply still arrives

Test the server without Unity

Swagger: 127.0.0.1:8000/docs

1. **uvicorn main:app --reload --port 8000**
2. Expand **POST /chat** → *Try it out*
3. Type a sentence → *Execute*
4. Send it without **text**: status **422**

Then open the audio

Copy **audio_url** from the reply into the browser after **127.0.0.1:8000**

If the browser plays it, Unity can download it

Symptom	Fix
uvicorn not found	The venv is not active
An echo of your own words	LIVE-4 not written, or no Ollama app

04

Unity-Python bridge

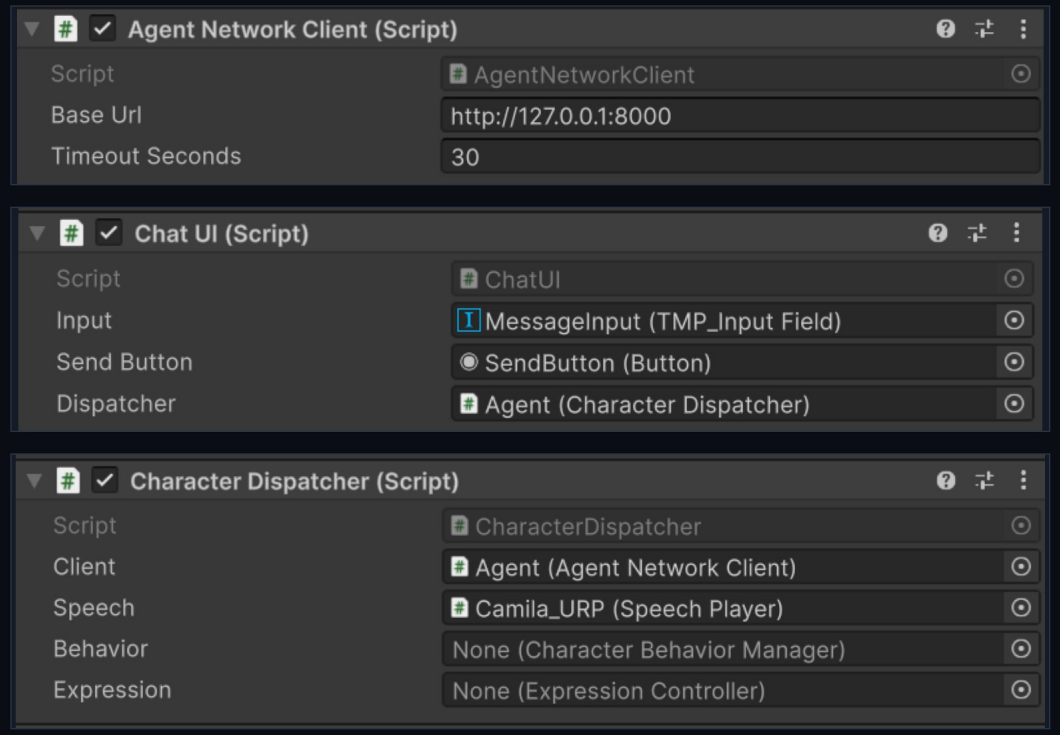
How does Unity send JSON?

What does the body do with the answer?

1. Unity sends **{ text }**
2. The server returns the contract
3. Unity downloads the WAV
4. SpeechPlayer: voice, lips, Speaking

Wire the typed chat in the scene

1. Hierarchy → **Create Empty** → name it **Network** → Add Component → **AgentNetworkClient**: Base Url **http://127.0.0.1:8000**, Timeout **30**
2. Select **Camila_URP** (root) → Add Component → **CharacterDispatcher**: Client = **Network**, Speech = **Camila_URP**; Behavior, Expression empty (webinar)
3. Hierarchy → UI → **Input Field - TextMeshPro**, then UI → **Button - TextMeshPro**: both land in a new **Canvas**. Button text = **Send**
4. Select **Canvas** → Add Component → **ChatUI**: Input = **InputField (TMP)**, Send Button = **Button**, Dispatcher = **Camila_URP**
5. Check: Edit → Project Settings → Player → Other Settings → **Allow downloads over HTTP** is **Always allowed** (already set in the starter)



AgentNetworkClient: one job

```
public class AgentNetworkClient : MonoBehaviour
{
    [SerializeField] string baseUrl = "http://127.0.0.1:8000";
    [SerializeField] int timeoutSeconds = 30;

    // POST /chat, a JSON body
    public IEnumerator Chat(string text,
        Action<ChatResponse> onOk, Action<string> onError)
    // POST /voice-chat, a form with the WAV
    public IEnumerator VoiceChat(byte[] wav,
        Action<ChatResponse> onOk, Action<string> onError)
    // shared by both: errors and parsing
    IEnumerator Send(UnityWebRequest req, ...)
}
```

Send a request, return the **reply** or an **error**

Coroutines: the game never freezes

127.0.0.1, not localhost

AgentNetworkClient: send JSON

```
public IEnumerator Chat(string text,
    Action<ChatResponse> onOk, Action<string> onError)
{
    string json = JsonUtility.ToJson(new ChatRequest { text = text });
    using (var req = new UnityWebRequest(baseUrl + "/chat", "POST"))
    {
        req.uploadHandler =
            new UploadHandlerRaw(Encoding.UTF8.GetBytes(json));
        req.downloadHandler = new DownloadHandlerBuffer();
        req.SetRequestHeader("Content-Type", "application/json");
        req.timeout = timeoutSeconds;
        yield return Send(req, onOk, onError);
    }
}
```

Not UnityWebRequest.Post(url, string): that sends form data

Raw body + **application/json**

A **timeout**: never wait forever

AgentNetworkClient: parse or fail cleanly

```
IEnumerator Send(UnityWebRequest req,
    Action<ChatResponse> onOk, Action<string> onError)
{
    yield return req.SendWebRequest();
    if (req.result ≠ UnityWebRequest.Result.Success)
    { onError?.Invoke(req.error); yield break; }

    ChatResponse response = null;
    try { response =
        JsonUtility.FromJson<ChatResponse>(req.downloadHandler.text); }
    catch (Exception e)
    { onError?.Invoke("Bad JSON: " + e.Message); yield break; }

    if (response == null)
    { onError?.Invoke("Empty response"); yield break; }
    onOk?.Invoke(response);
}
```

Network error → **onError**

Not JSON → **onError**, no exception

Allow downloads over HTTP: set in the setup

LIVE-5: download the voice, then speak

```
public IEnumerator Handle(ChatResponse r)
{
    AudioClip clip = null;
    // ===== LIVE-5a · Network ▸ Voice =====
      if (!string.IsNullOrEmpty(r.audio_url))
          yield return speech.Download(Resolve(r.audio_url), c ⇒ clip = c);
      else
          Debug.LogWarning("TTS failed: no audio for this reply.");

    // ===== TURN-7 (webinar): face and gesture, here =====

    // ===== LIVE-5b · Voice ▸ Body =====
      if (clip ≠ null) yield return speech.Play(clip);
      else if (behavior ≠ null) yield return new WaitForSeconds(1.5f);
}
```

Download **first**, then play: voice and body start together

speech.Play: your TURN-1 does the rest

The empty block waits for the webinar

TURN-3: one typed turn

```
public void Say(string text)            // ChatUI calls this
{
    if (Busy || string.IsNullOrWhiteSpace(text)) return;
    StartCoroutine(SayRoutine(text));
}

IEnumerator SayRoutine(string text)
{
    Busy = true;
      // ===== TURN-3 · UI ▸ Network ▸ Body =====
      // ask client.Chat(text, onOk, onError),
      // keep the reply or the error, then:
      // a reply → Handle(reply)   an error → Fail(error)
    Busy = false;
}
```

You need: **client.Chat**, two lambdas, **Handle**, **Fail**

yield return each step: one after the other

The scene is already wired: **Network**, dispatcher, chat box

TURN-3: solution

CharacterDispatcher.cs

```
public void Say(string text)              // ChatUI calls this
{
    if (Busy || string.IsNullOrWhiteSpace(text)) return;
    StartCoroutine(SayRoutine(text));
}

IEnumerator SayRoutine(string text)
{
    Busy = true;
    // ===== TURN-3 · UI ▸ Network ▸ Body =====
      ChatResponse reply = null; string error = null;
      yield return client.Chat(text, r ⇒ reply = r, e ⇒ error = e);
      if (reply ≠ null) yield return Handle(reply);
      else yield return Fail(error);
    Busy = false;
}
```

Try it: the first round trip

1. Server running in a terminal
2. Unity: Play, type a sentence, **Send**
3. The character speaks the answer

The first **end-to-end** moment: text → Python → LLM → voice → body

Screenshot taken with the **echo brain**: with the Ollama app running she answers in her own words

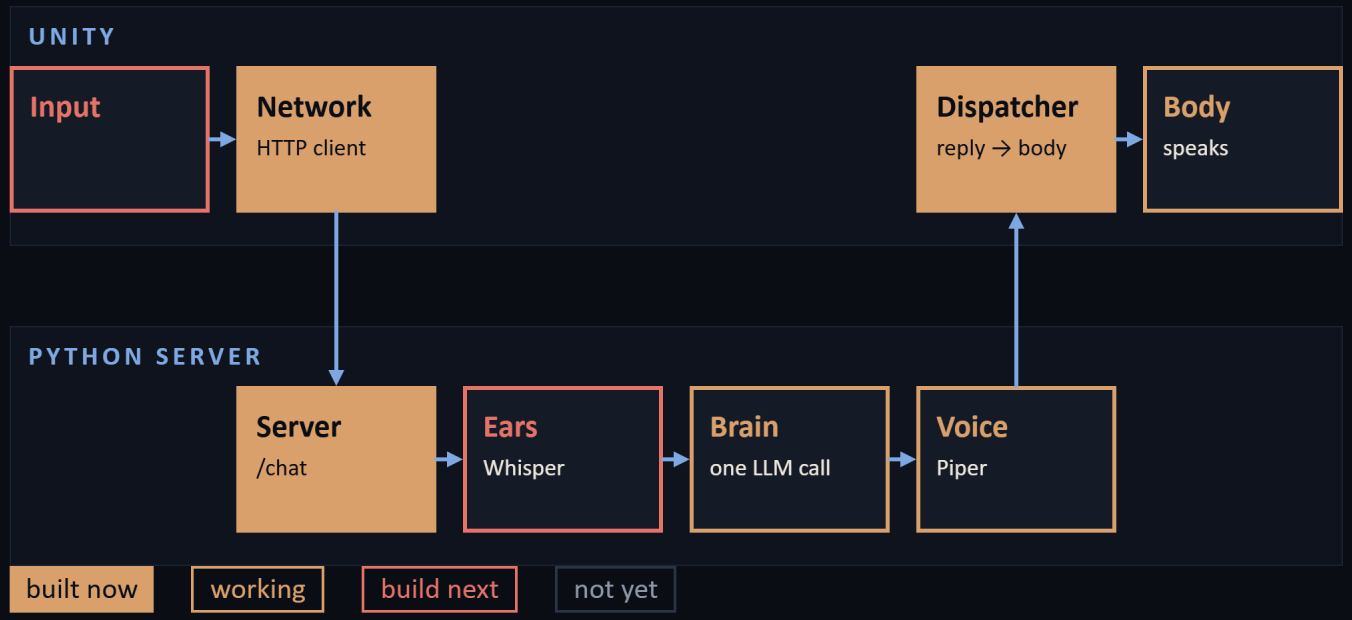


When things fail

Failure	What the character does
Server down or timeout	Stays idle; the error is logged (homework: an in-character line)
Bad JSON	Log it, ignore the turn
LLM fails or quota used up	The server answers with the echo: the loop never breaks
Missing audio	Nothing to play: idle, error logged
Slow reply	Something to show while waiting (with the voice loop)

A virtual human that freezes when the server is slow is worse than no virtual human.

What works now: the first round trip



✓ **Server**
/docs answers; the WAV plays in the browser

✓ **Client**
Unity parses the reply, errors reported cleanly

✓ **Round trip**
Send → the character speaks the answer

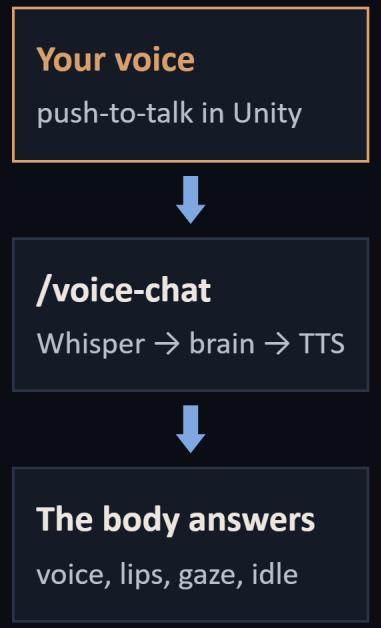
Next: the microphone.

05

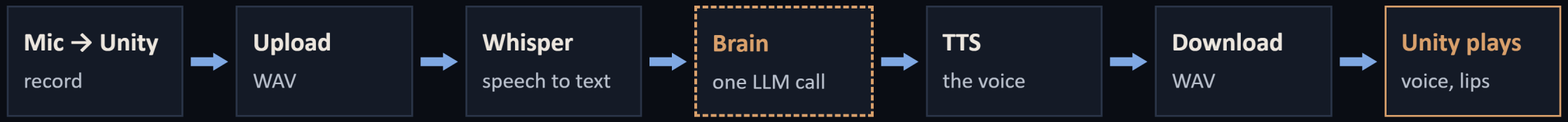
Voice in

What changes when you speak instead of type?

Where does the time go?



The whole loop and its latency budget



THE BRAIN text only, a neutral body · structured JSON in the webinar · LangGraph in Part 2

Most parts exist already: today adds the **first two** boxes

Every stage adds delay: we **measure** each one

Greta 2.0 aims at turn gaps of a few hundred milliseconds: today we measure our baseline

MicRecorder 1/3: push-to-talk

```
using System.IO;
using System.Text;
using UnityEngine;

public class MicRecorder : MonoBehaviour
{
    [SerializeField] int sampleRate = 16000;
    [SerializeField] int maxSeconds = 20;
    AudioClip clip;
    string device;
    public bool IsRecording { get; private set; }

    public void StartRecording()
    {
        if (Microphone.devices.Length == 0)
        { Debug.LogError("No microphone"); return; }
        device = Microphone.devices[0];
        clip = Microphone.Start(device, false, maxSeconds, sampleRate);
        IsRecording = true;
    }
```

Hold a key to record, **release** to send

The character never hears itself

16 kHz mono: what Whisper expects

MicRecorder 2/3: stop and cut

```
// Returns a WAV file (bytes), or null if nothing was recorded.
public byte[] StopRecording()
{
    if (!IsRecording) return null;
    IsRecording = false;
    int samples = Microphone.GetPosition(device);
    Microphone.End(device);
    if (samples < sampleRate / 4) return null;  // under 0.25 s

    var data = new float[samples * clip.channels];
    clip.GetData(data, 0);
    return ToWav(data, clip.channels, sampleRate);
}
```

Cut to the length **actually recorded**

A quick tap is ignored

Encoded as WAV **in memory**

MicRecorder 3/3: a WAV in memory

```
    static byte[] ToWav(float[] samples, int channels, int rate)
    {
        using (var ms = new MemoryStream())
        using (var w = new BinaryWriter(ms))
        {
            int bytes = samples.Length * 2;
            w.Write(Encoding.ASCII.GetBytes("RIFF")); w.Write(36 + bytes);
            w.Write(Encoding.ASCII.GetBytes("WAVE"));
            w.Write(Encoding.ASCII.GetBytes("fmt ")); w.Write(16);
            w.Write((short)1); w.Write((short)channels);
            w.Write(rate); w.Write(rate * channels * 2);
            w.Write((short)(channels * 2)); w.Write((short)16);
            w.Write(Encoding.ASCII.GetBytes("data")); w.Write(bytes);
            foreach (var s in samples)
                w.Write((short)(Mathf.Clamp(s, -1f, 1f) * short.MaxValue));
            w.Flush();
            return ms.ToArray();
        }
    }
}
```

A **44-byte header**

Then 16-bit samples

Provided: read it, don’t type it

Unity: upload the audio

```
// AgentNetworkClient.cs (provided, next to Chat)
public IEnumerator VoiceChat(byte[] wav,
    Action<ChatResponse> onOk, Action<string> onError)
{
    var form = new WWWForm();
    form.AddBinaryData("audio", wav, "speech.wav", "audio/wav");
    using (var req = UnityWebRequest.Post(baseUrl + "/voice-chat", form))
    {
        req.timeout = timeoutSeconds;
        yield return Send(req, onOk, onError);
    }
}
```

A file upload = **multipart form data**

Here **Post** is right: it is a form

Send from the bridge, reused

06

The whole loop

What does the body do while it waits?

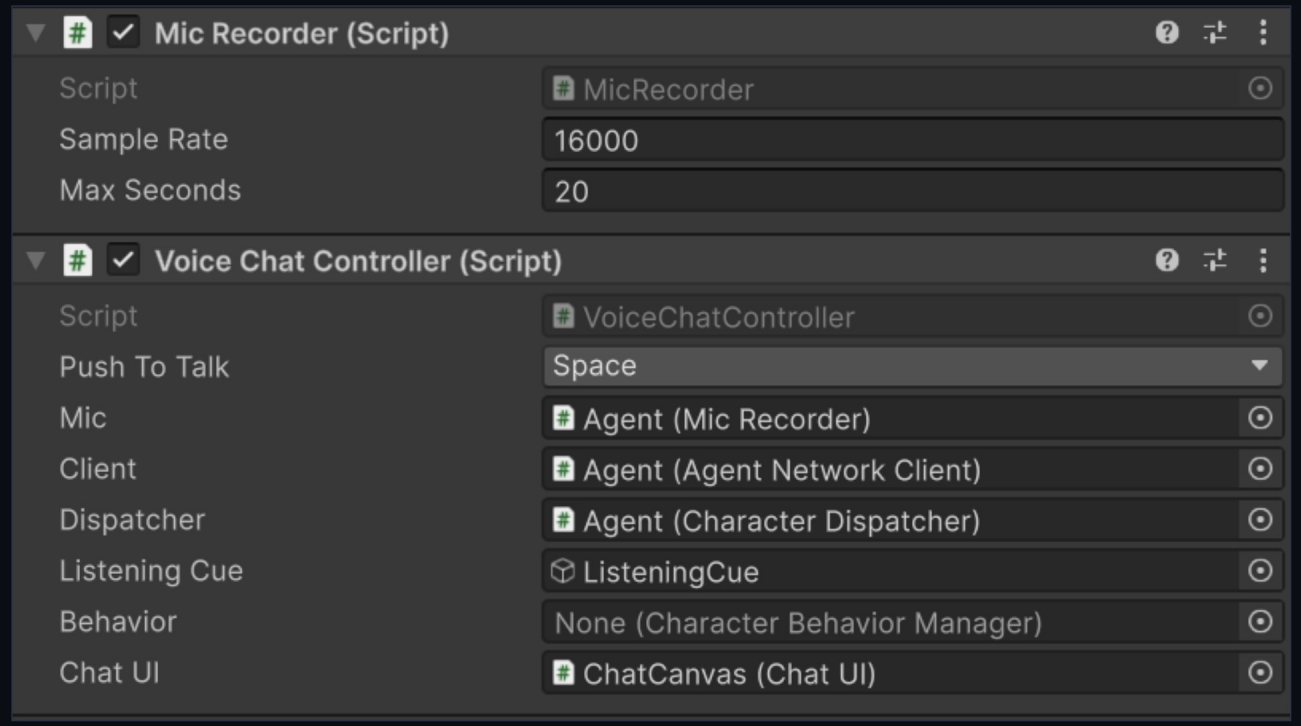
How long does one turn take?

Speaks voice and lips, the Speaking layer	`SpeechPlayer`
Looks at you look-at and saccades	`GazeController`
Alive breathing, blinking	`Idle + Blink`
Gestures, emotions chosen by the brain	`webinar`

Wire the voice loop in the scene

1. Select **Network** → Add Component → **MicRecorder** (keep 16000 Hz, 20 s)
2. Right-click **Canvas** → UI → **Text - TextMeshPro** → name it **ListeningCue**, text ● **Listening…**, then untick it: hidden at start
3. Select **Network** → Add Component → **VoiceChatController**: fill it as in the screenshot

macOS: allow the **microphone** for Unity when the OS asks



VoiceChatController: hold to talk

```
public class VoiceChatController : MonoBehaviour
{
    [SerializeField] KeyCode pushToTalk = KeyCode.Space;
    [SerializeField] MicRecorder mic;
    [SerializeField] AgentNetworkClient client;
    [SerializeField] CharacterDispatcher dispatcher;
    [SerializeField] GameObject listeningCue;     // optional icon
    [SerializeField] CharacterBehaviorManager behavior; // webinar
    [SerializeField] ChatUI chatUI;               // no talking while typing

    void Update()
    {
        if (busy || (chatUI ≠ null && chatUI.IsTyping)) return;
        // key down: StartRecording, show the cue, grey the chat
        // key up:   StopRecording → a WAV? StartCoroutine(Run(wav))
    }
```

Space by default

busy: one turn at a time

Run(wav): the turn we write

LIVE-6: one spoken turn

```
IEnumerator Run(byte[] wav)
{
    busy = true;  State = "Thinking";
    // ===== LIVE-6 · Input ▸ Network ▸ Body =====
      ChatResponse reply = null; string error = null;
      yield return client.VoiceChat(wav, r ⇒ reply = r, e ⇒ error = e);
      Show(false);
      if (reply ≠ null)
      {
          Debug.Log($"You said: {reply.transcript}");
          State = "Speaking";
          yield return dispatcher.Handle(reply);    // the TURN-3 door
      }
      else yield return dispatcher.Fail(error);
    busy = false;  State = "Idle";
}
```

The same shape as **TURN-3**: ask, then Handle or Fail

The cue stays on while the server **listens and thinks**

Nothing new in the body: the same **Handle** as TURN-3

TURN-4: the server hears

```python
@app.post("/voice-chat", response_model=ChatResponse)
async def voice_chat(audio: UploadFile = File(...)):
    # ===== TURN-4 · Server ▸ Ears ▸ respond =====
    # path = save_upload(audio)
    # transcribe(path)  -- it blocks: run_in_threadpool
    # time it (t_stt_ms), delete the file
    # nothing heard → respond("please repeat"...)
    # else → respond(transcript, transcript, t_stt_ms)
    return await respond("Voice chat is not connected yet.")
```

Mirror **/chat**: one box more in front, the **ears**

save_upload and **transcribe** are provided

Test in **/docs** with your **Recording.m4a** before Unity

TURN-4: solution

main.py

```python
@app.post("/voice-chat", response_model=ChatResponse)
async def voice_chat(audio: UploadFile = File(...)):
    # ===== TURN-4 · Server ▸ Ears ▸ respond =====
    path = save_upload(audio)
    t0 = time.perf_counter()
    transcript = await run_in_threadpool(transcribe, path)
    t_stt = int((time.perf_counter() - t0) * 1000)
    Path(path).unlink(missing_ok=True)
    print(f"Heard: {transcript!r}")
    if not transcript:
        return await respond("Sorry, I did not catch that. Could you repeat?", "", t_stt)
    return await respond(transcript, transcript, t_stt)
```

Try it: talk to your character

Server running → Play → **click the Game view** → hold **Space** and speak

Say	Expect
“Hello!”	a greeting back
“I failed my exam today.”	kind words, a neutral face
“Tell me a joke.”	a short joke, a blank body



Measure the voice loop

Already in the Console

```
[llm] stt 412 ms | brain 1830 ms | tts 290 ms
```

The dispatcher logs the server's **timings_ms** for every reply: nothing to write.

Three sentences: your numbers

Stage	ms
Ears (Whisper)	
Brain (one LLM call)	
Voice (Piper)	
Total, until the voice starts	

Webinar: what does **structured JSON** add on top?

Next: give the body a meaning

You have

- ✓ A body: idle, blink, gaze, speaking layer, lips
- ✓ A brain: one LLM call via the Ollama app
- ✓ Ears and voice: Whisper and Piper
- ✓ A bridge, and a voice loop

Webinar, online

Gestures and emotions on the body

The same LLM call, structured JSON

A body that means what it says

Unity: the body → **FastAPI: the bridge** → **The brain is whatever you plug in**

What works now: the voice loop



✓ **The loop**

Hold Space, speak: the character answers

✓ **Timings**

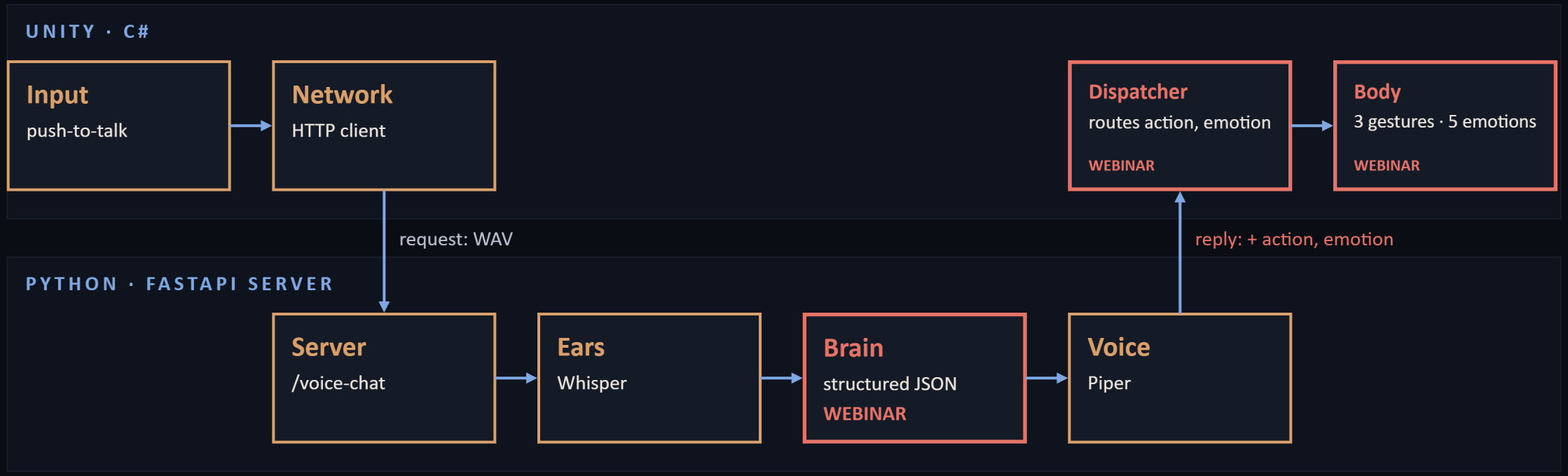
Three sentences measured

✓ **For the webinar**

This loop working, the Ollama app signed in, the Gesture layer built

NEXT · WEBINAR, ONLINE

In the webinar: a body that means what it says



working · build next

CONTRACT V2 `{ "speech_text", "audio_url", "action": "WAVE", "emotion": "HAPPY" }`

Thank you

Questions?

Next · **the webinar, online**: gestures, emotions and a brain that chooses them



Stefano Calzolari · Wed 21 – Fri 23 October 2026